

LABMENTIX

Cybersecurity Training Program

CYBERSECURITY PROJECT REPORT

15-Day Hands-On Project Submission

Project Title	[Caesar and Vigenere Cipher Tool]
Student Name	[Mohammed Eshaan]
Batch / Cohort	[1 September 2026 Cybersecurity batch]
Domain	[Cryptography / Information security]
Trainer Name	Ganesh Kuche
Submission Date	[22 / 07/ 2026]
Difficulty Level	[Beginner / Intermediate / Advanced]

SECTION 1 — PROJECT ABSTRACT

1. Project Abstract

Classical ciphers are simple to implement but are also the standard first lesson in why weak cryptography fails. This project presents Cipher Desk, a single-file web tool (HTML, CSS and vanilla JavaScript) that encrypts, decrypts and breaks the Caesar and Vigenère ciphers entirely in the browser, with no server or external libraries. Caesar messages are broken by trying all 26 shifts and ranking them with a chi-squared test against English letter frequencies. Vigenère messages are broken by estimating the key length with the Index of Coincidence and then solving each key position as its own Caesar cipher. Testing showed that all encryption and decryption round-trips were exact, and that the Vigenère breaker recovered the plaintext in 6 of 6 trials once the ciphertext reached about 500 letters, but was unreliable below roughly 300 letters.

SECTION 2 — INTRODUCTION

2. Introduction

2.1 Background

Cryptography protects the confidentiality of information by transforming readable text (plaintext) into unreadable text (ciphertext) using a key. Cryptanalysis is the opposite discipline: recovering the message or key without being told it. The Caesar cipher (a fixed letter shift) and the Vigenère cipher (a repeating keyword of shifts) are historical ciphers that are no longer secure, yet they clearly demonstrate the core ideas of key space, frequency analysis and statistical attacks that still shape how modern ciphers are designed and evaluated.

2.2 Problem Statement

Learners often read about frequency analysis and the Index of Coincidence without seeing them work on real input. Existing online cipher tools typically only encrypt and decrypt, need a known key, or send the text to a server. This project addresses the gap by providing one offline, self-contained tool where the same text can be encrypted with a chosen key and then attacked automatically without the key, so the strength and weakness of each cipher can be observed side by side.

2.3 Objectives

- [Objective 1 — To implement Caesar and Vigenère encryption and decryption that preserve letter case, spaces, digits and punctuation.]
- [Objective 2 — To build an automatic Caesar breaker using chi-squared scoring against English letter frequencies.]
- [Objective 3 — To build an automatic Vigenère breaker using the Index of Coincidence for key-length estimation and per-column frequency analysis for key recovery.]
- [Objective 4 — To design a clear, responsive interface with six modes (encrypt, decrypt and break for each cipher) and one-click copy of results.]
- [Objective 5 — To test correctness and measure how reliable the breakers are as the ciphertext gets shorter.]

SECTION 3 — TOOLS & ENVIRONMENT

3. Tools and Environment Setup

Tool / Software	Version / Platform	Purpose
[HTML5 / CSS3]	[Web standards]	[Page structure, layout, responsive design (CSS grid and flexbox)]
[JavaScript (vanilla, ES6)]	[Runs in any modern browser]	[All cipher logic, statistical analysis and UI wiring]
[Web browser]	[Version 153.0.4234.32]	[Running and manually testing the tool]
[Google Fonts (Spectral, Spectral SC, JetBrains Mono)]	[Loaded via CSS @import]	[Typography; falls back to Georgia and system monospace when offline]
[Text editor]	[Version: 1.137.0]	[Writing the single HTML file]
[Node.js]	[Version: 24.18.1 V8: 14.8.178.38-electron.0]	[Running the core functions outside the browser to run repeatable tests]

3.1 Lab / Environment Description

The whole project is one file, index.html, containing the markup, the stylesheet and the script. It needs no build step, framework, package manager or backend: opening the file in a browser is enough to run it. All processing happens on the user's device, so no plaintext, ciphertext or key is ever sent over the network (the only outside request is the optional font download). Development and testing were done on [your operating system and machine, e.g. Windows 11 laptop].

SECTION 4 — METHODOLOGY / STEP-BY-STEP PROCESS

4. Methodology

Step 1 — Plan the tool and choose the algorithms

I decided on six modes: Encrypt, Decrypt and Break for both Caesar and Vigenère, grouped in a sidebar. The rule for every mode is the same: only the letters A–Z are changed, while case, spaces, digits and punctuation pass through untouched. For the two "break" modes I chose statistical methods that need no dictionary: chi-squared scoring for Caesar and the Index of Coincidence for Vigenère. Step 2 — Build the character-shifting function Every cipher in the tool is built on one function that shifts a single character. The shift is normalised with a double modulo so negative values (used for decryption) also work, and upper- and lower-case letters are handled separately.

Step 2 — Build the character-shifting function

Every cipher in the tool is built on one function that shifts a single character. The shift is normalised with a double modulo so negative values (used for decryption) also work, and upper- and lower-case letters are handled separately.

```
function caesarShiftChar(ch, shift){  
  const s = ((shift % 26) + 26) % 26;  
  const code = ch.charCodeAt(0);  
  if (ch >= 'A' && ch <= 'Z')  
    return ALPHABET[(code - 65 + s) % 26];  
  else if (ch >= 'a' && ch <= 'z')  
    return ALPHABET[(code - 97 + s) % 26].toLowerCase();  
  return ch; // non-letters unchanged  
}
```

Step 3 — Implement Caesar encrypt and decrypt

Encryption maps every character through caesarShiftChar with the chosen shift (0–25). Decryption simply calls encryption with the negative shift, so there is only one piece of logic to verify.

```
function caesarEncrypt(text, shift){  
  return [...text].map(ch => caesarShiftChar(ch, shift)).join("");  
}  
function caesarDecrypt(text, shift){  
  return caesarEncrypt(text, -shift);  
}
```

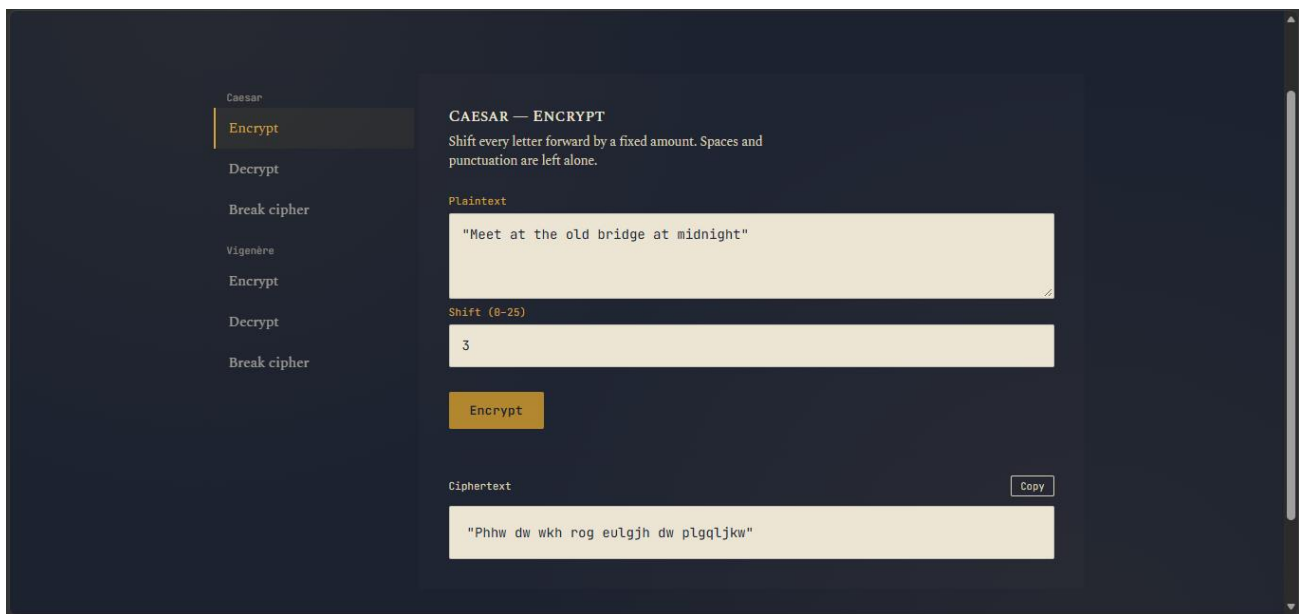


Figure 1: Caesar Encrypt tab: plaintext "Meet at the old bridge at midnight" with shift 3 Produces "Phhw dw wkh rog eulgjh dw plgqljkw".

Step 4 — Build the Caesar breaker (chi-squared scoring)

With only 26 possible keys, the breaker decrypts the text with every shift and scores each candidate. The score compares how often each letter actually appears with how often it would appear in English (for example E about 12.7%, T about 9.1%). The lower the chi-squared value, the closer the text is to English, and the lowest-scoring shift is reported.

```
for (const letter of ALPHABET){
  const observed = counts[letter] || 0;
  const expected = ENGLISH_FREQ[letter] / 100 * n;
  score += (observed - expected) ** 2 / expected;
}
// breakCaesar(): try shift 0..25, keep the candidate with the lowest score
```

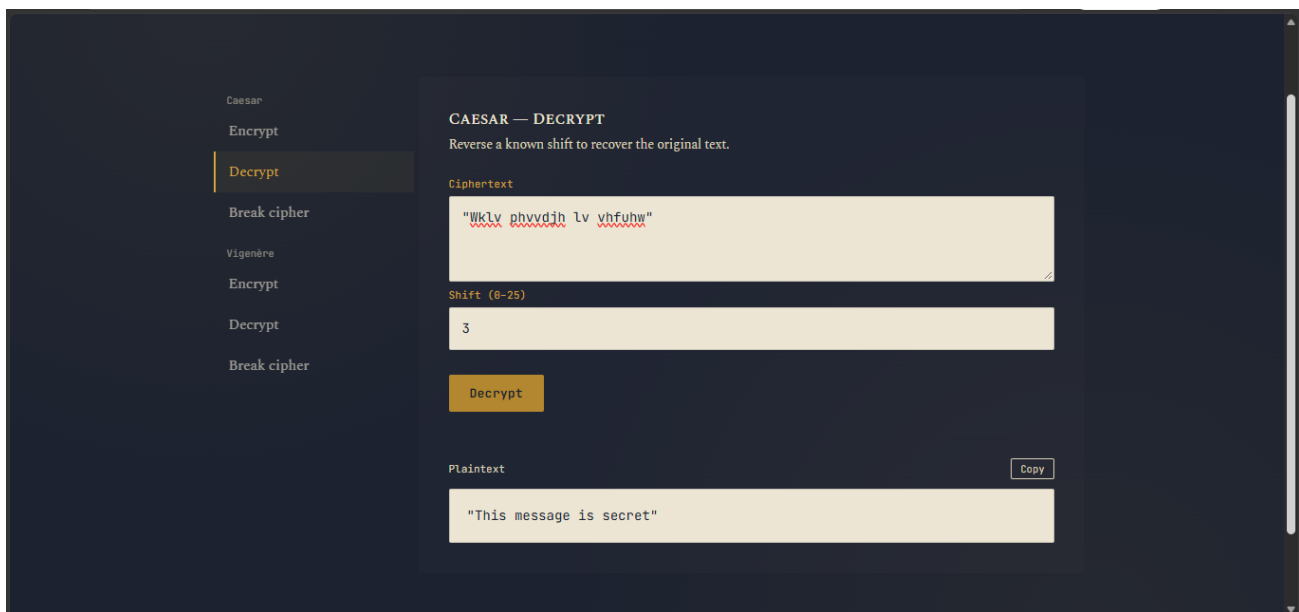


Figure 2: Caesar Break tab recovering "This message is secret" from "Wklv phvvdjh lv vhfuhw" and reporting a detected shift of 3

Step 5 — Implement Vigenère encrypt and decrypt

The keyword is cleaned to letters only and converted to upper case. Each plaintext letter is shifted by the next key letter, and the key position advances only on letters, so spaces and punctuation do not consume key characters. Decryption builds an "inverse key" (each key letter replaced by 26 minus its value) and reuses the encrypt function.

```
const shift = key.charCodeAt(keyIndex % key.length) - 65;  
result += caesarShiftChar(ch, shift);  
keyIndex++; // only advanced for letters
```

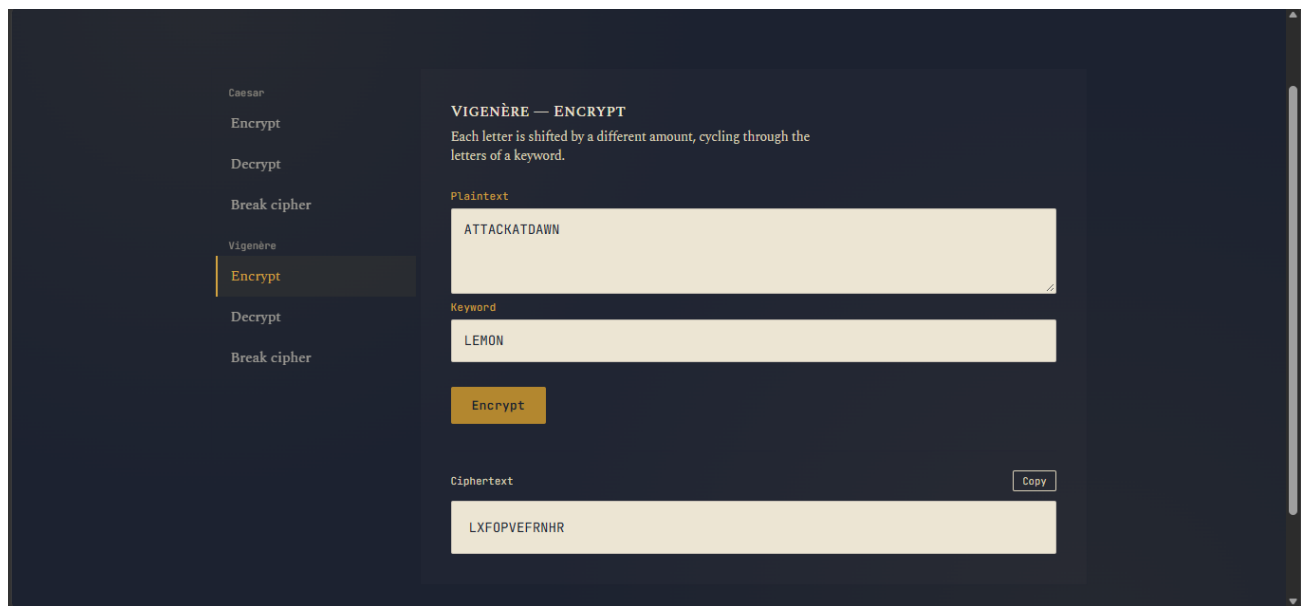


Figure 3: Vigenère Encrypt tab with keyword LEMON. Verified with the standard test vector: ATTACKATDAWN becomes LXFOPVEFRNHR.

Step 6 — Estimate the Vigenère key length with the Index of Coincidence

The Index of Coincidence (IC) is the probability that two letters picked at random from a text are the same: $IC = S f(f-1) / (N(N-1))$. English text scores about 0.067, while random letters score about 0.038. For each candidate key length k , the ciphertext is split into k columns (every k -th letter). If k is right, each column is just a Caesar-shifted piece of English and has a high IC. The tool averages the IC of the columns for $k = 1$ up to the user-set maximum and picks the k with the highest average.

```
lettersOnly.forEach((c, idx) => groups[idx % k].push(c)); // split into k columns
const avgloc = mean(groups.map(g => indexofCoincidence(g)));
if (avgloc > bestAvgloc){ bestAvgloc = avgloc; bestLen = k; }
```

Step 7 — Recover the key letters and decrypt

Once the key length is known, each column is a plain Caesar cipher, so the tool reuses breakCaesar on every column. The shift found for column i is the i -th key letter. Finally the recovered key is passed to vigenereDecrypt and both the key and the plaintext are displayed.

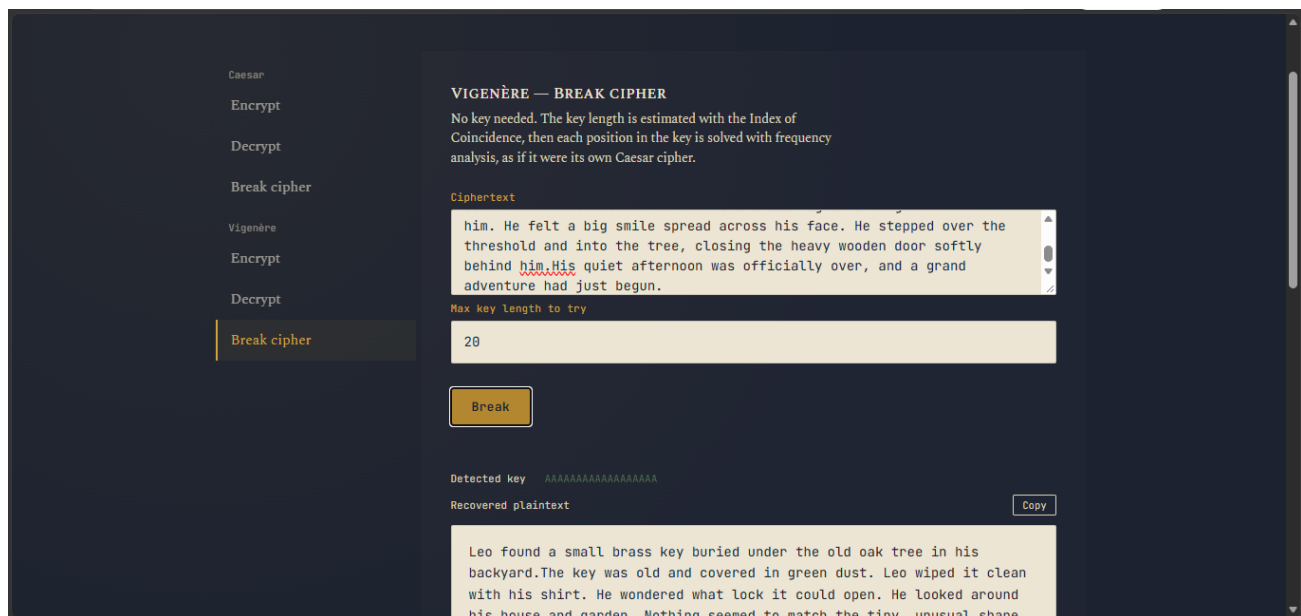


Figure 4: Vigenère Break tab showing the detected key and the recovered plaintext for a long ciphertext.

Step 8 — Build the interface

The layout is a two-column desk: a sidebar of tabs on the left and the active panel on the right, which collapses to one column on screens narrower than 720 px. A small script shows one pane at a time. Results are written with `textContent`, not `innerHTML`, so user text can never be interpreted as HTML. Each output has a Copy button that uses the browser clipboard API.

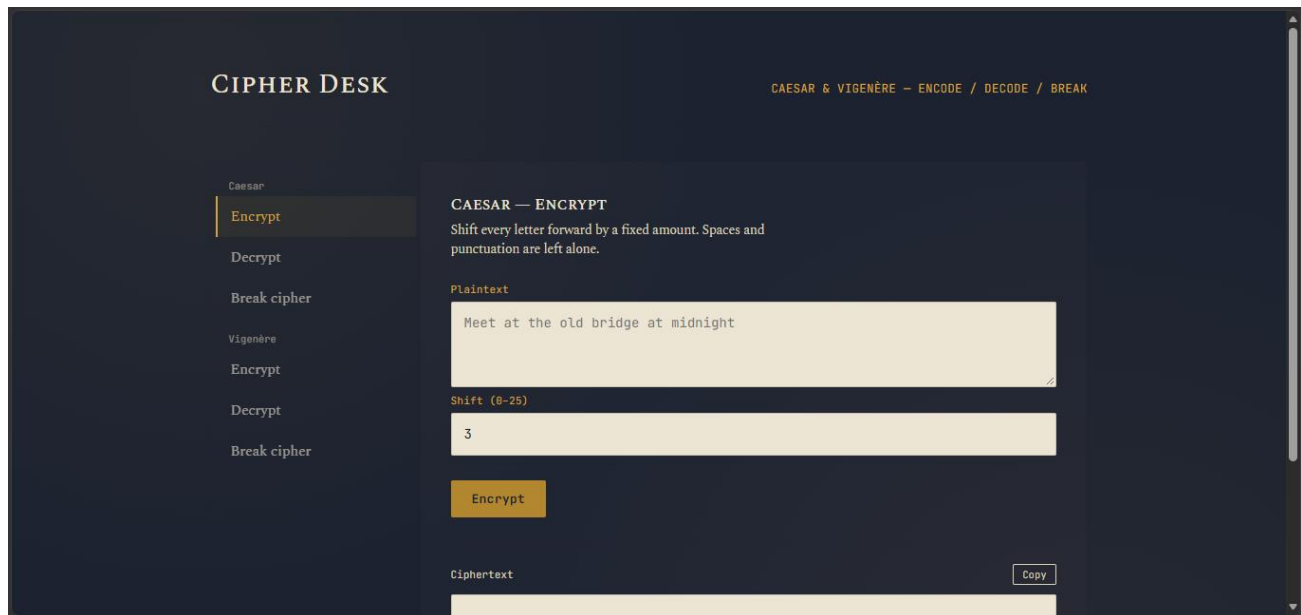


Figure 5: The full Cipher Desk interface (sidebar tabs and active panel) in a desktop browser window.

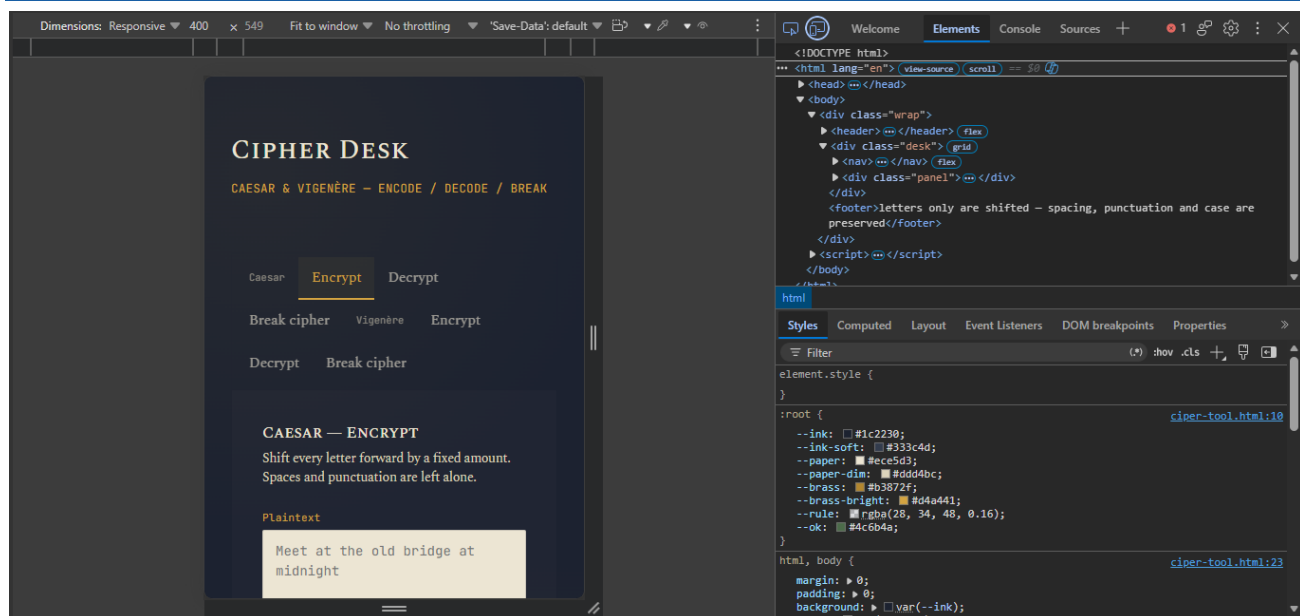


Figure 6: The same interface on a narrow (mobile-width) window, showing the tabs wrapping above the panel.

Step 9 — Test correctness and breaker reliability

I copied the core functions into a Node.js script and ran repeatable tests: known examples, round-trips, and the two breakers on a long English passage (the opening paragraph of A Tale of Two Cities, 578 letters) encrypted with six different keys and cut to different lengths.

```
node test.js
Vig enc LEMON: LXFOPEFRNHR      (ATTACKATDAWN, matches the standard example)
rt caesar 0/1/13/25: true        (decrypt(encrypt(x)) == x)
rt vig ("Key-Word!"): true
break caesar, shifts 3/7/19: recovered correctly
```

SECTION 5 — RESULTS & OBSERVATIONS

5. Results and Observations

5.1 Summary of Findings

#	Finding / Observation	Details	Severity / Impact
1	Encryption and decryption are exact	Caesar with shifts 0, 1, 13 and 25, and Vigenère with key "Key-Word!", returned the original text on a mixed sentence containing capitals, commas, a semicolon, digits and "!". The standard vector ATTACKATDAWN + LEMON gave LXFOPVEFRNHR.	Info (pass)
2	Caesar breaker works on very short text	Correct shift found for a 28-letter sentence at shifts 3, 7 and 19, for the 12-letter "Attack at dawn" at shift 5, and for "Wklv phvvdjh lv vhfuhw" (shift 3). Only a handful of samples were tried, so short texts may still fail.	Info (pass)
3	Vigenère breaker needs a long ciphertext	On six keys (LEMON, CIPHER, KEY, SECURITY, SHIELD, VAULT): 0/6 correct at 100 letters, 1 to 3 of 6 at 150–250, 4/6 at 300–400, 5/6 at 450 and 6/6 at 500 and 578 letters (Table 5.2).	Medium (limitation)
4	Detected key is often a repeated multiple of the real key	LEMON was reported as LEMONLEMONLEMONLEMON (length 20), CIPHER as CIPHERCIPHER, KEY as KEYKEYKEYKEY, SECURITY as SECURITYSECURITY. The plaintext was still 100% correct, because a repeated key decrypts identically, but the displayed key is not the shortest form.	Low
5	IC clearly shows the true key length	For LEMON (length 5) the average IC was 0.073 at $k = 5$ and about 0.077, 0.076 and 0.081 at $k = 10, 15$ and 20, against roughly 0.041–0.049 at all other lengths. Multiples of the true length also peak, which explains finding 4.	Info
6	Privacy and input handling	All work is done locally; no request carries user text. Results use textContent. The two innerHTML uses only insert a shift number and a key built solely from the letters A–Z, so there is no injection path.	Info (positive)

7	Both ciphers are trivially weak against the tool itself	Caesar has just 26 keys and was broken instantly. Vigenère fell to statistics once enough text was available, confirming why neither is used for real security.	Info
---	---	---	------

5.2 Analysis

The results match the theory. Encryption and decryption are simple modular arithmetic, so they were exact. The Caesar breaker works even on short inputs because only 26 candidates compete and English letter frequencies are strongly uneven. The Vigenère breaker is harder because it stacks two estimates. First it must pick the right key length from IC values, then each column must be long enough for frequency analysis. A 100-letter ciphertext with a 5-letter key leaves only about 20 letters per column, which is too few to trust letter counts. Short columns also make each IC value noisy, which likely explains the uneven results at 150–250 letters and the occasional wrong key. The repeated-key behaviour (finding 4) follows from the same statistics: if a key of length 5 works, then lengths 10, 15 and 20 also split the text into columns that are each single-shift, so they score highly too. This links directly to what was studied in the sessions: the weakness of a repeating key is exactly what Kasiski examination and IC exploit.

SECTION 6 — CHALLENGES FACED

6. Challenges Faced

Challenge Faced	How I Solved It
Keeping case, spaces, digits and punctuation intact while shifting letters	Handled upper- and lower-case ranges separately inside one function and returned every other character unchanged.
Making the Vigenère key line up correctly when the text has spaces and punctuation	Advanced the key index only when a character is a letter, and stripped non-letters from the keyword itself.
Avoiding duplicate (and possibly inconsistent) logic for decryption	Decrypted Caesar by negating the shift and Vigenère by building an inverse key and reusing the encrypt function.
Choosing how to rank the 26 Caesar candidates	Used a chi-squared score against standard English letter frequencies and picked the minimum.
Vigenère breaker returning wrong keys on short ciphertext and repeated keys on long ciphertext	Measured the behaviour by testing several keys and lengths, documented the minimum useful length (about 500 letters), and noted the limitation on the page ("the longer the text, the more reliable the guess").

SECTION 7 — LEARNING OUTCOMES

7. Learning Outcomes

- [Learning outcome 1 - I learned how a shift cipher works as modular arithmetic on the alphabet, and why decryption is the same operation with a negative key.]
- [Learning outcome 2 - I learned how to score a candidate plaintext with a chi-squared test against English letter frequencies, and why this is enough to break a 26-key cipher.]
- [Learning outcome 3 - I learned what the Index of Coincidence measures (about 0.067 for English versus about 0.038 for random text) and how splitting text into columns exposes a repeating key length.]
- [Learning outcome 4 - I learned that a statistical attack depends on sample size: the same breaker that works on 500 letters fails on 100, and that measuring this is part of evaluating any attack.]
- [Learning outcome 5 - I learned to write and test a browser tool without dependencies, including safe output handling with `textContent` and a responsive layout with CSS grid and media queries.]
- [Learning outcome 6 - I learned why repeated-key ciphers are weak and how this motivates one-time pads and modern ciphers such as AES.]

SECTION 8 — CONCLUSION & FUTURE SCOPE

8. Conclusion

Cipher Desk met all five objectives: it encrypts, decrypts and automatically breaks both Caesar and Vigenère ciphers in a single offline file with a clean six-mode interface. Encryption and decryption were verified exact, the Caesar breaker worked on every short sample tried, and the Vigenère breaker recovered the plaintext in every trial from 500 letters upward. The main weakness is reliability on short Vigenère ciphertext and the habit of reporting a repeated form of the key. Overall the project shows in practice why classical ciphers cannot protect real data and how little a statistical attacker needs.

8.1 Future Scope

- Report the shortest key length. A quick experiment that picked the smallest length within 90% of the peak IC returned the true length for 4 of 6 keys but double the true length for the other 2 (LEMON and VAULT), so a better rule (or testing divisors of the detected length) is needed.
- Add Kasiski examination as a second key-length estimate and show the IC chart for each length so users can see the peaks.
- Show the top few candidate keys or shifts with their scores, instead of only the best one.
- Support other languages and letter-frequency tables, and handle accented letters.
- Add more ciphers (Atbash, Playfair, XOR) and a small library of automated tests inside the repository.

SECTION 9 — REFERENCES

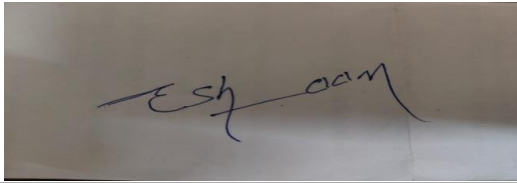
9. References

- [1] Caesar cipher — https://en.wikipedia.org/wiki/Caesar_cipher
- [2] Vigenère cipher — https://en.wikipedia.org/wiki/Vigen%C3%A8re_cipher
- [3] Index of coincidence — https://en.wikipedia.org/wiki/Index_of_coincidence
- [4] Letter frequency — https://en.wikipedia.org/wiki/Letter_frequency
- [5] Chi-squared test — https://en.wikipedia.org/wiki/Chi-squared_test
- [6] Kasiski examination — https://en.wikipedia.org/wiki/Kasiski_examination
- [7] MDN Web Docs (HTML, CSS, JavaScript, Clipboard API) — <https://developer.mozilla.org>
- [8] Google Fonts: Spectral and JetBrains Mono — <https://fonts.google.com>
- [9] Test text: opening paragraph of A Tale of Two Cities by Charles Dickens (public domain)

DECLARATION

Student Declaration

I hereby declare that this project report is my own original work, completed as part of the Labmentix Cybersecurity Training Program. The work presented in this report is genuine, and all external references have been duly acknowledged.

A photograph of a handwritten signature in black ink on a light-colored surface. The signature appears to be 'Eshaan'.

Student Signature

22 September 2026

Date of Submission